

Benjamin Tran-Pugh

647-768-1669 | btranpug@uwaterloo.ca | [linkedin.com/in/benjamin-tran-pugh](https://www.linkedin.com/in/benjamin-tran-pugh) | github.com/BenjaminTP | benjamintranpugh.com

EDUCATION

University of Waterloo

Candidate for BASc, Mechanical Engineering (Co-op)

Cumulative GPA: 3.83/4.0

Sep 2025 – Jun 2030

Waterloo, ON

TECHNICAL SKILLS

CAD & Analysis: SolidWorks (FEA, PDM, Sheet Metal), Onshape (FEA), Fusion 360, Autodesk Inventor, AutoCAD, GD&T

Software: Python (NumPy, SciPy, Matplotlib), C/C++ (ESP32), MATLAB, Linux (Raspberry Pi, Jetson Orin Nano), Git

Other: 3D Printing (PLA, PETG, TPU, ABS, PA6-CF), DFM/DFA, Machining, Technical Drawings, Microsoft Suite, Soldering

EXPERIENCE

Mechanical Engineering Intern

Human Computer Lab (SPC '25)

Feb 2026 – Aug 2026

Toronto, ON

- Owned mechanical and actuation development of LeLamp, an animatronic robotics product, across three actuator generations, advancing it from functional prototype toward manufacturing readiness through EVT/DVT stages.
- Led R&D efforts in building animated robots from scratch, integrating impedance control, quiet operating noise, and led testing to determine safety margins on torque, overheating, noise, and vibrations against simulation results
- Designed and hardware-validated a spring-based gravity-compensation system to eliminate actuator thermal overload, deriving joint balance conditions from Lagrangian mechanics and verifying against published literature
- Designed mechanisms such as antagonistic pairs, capstan drive, and full/partial spring gravity compensation by running through calculations, simulating physics, testing real-world results against simulations, and iterating
- Identified root causes of motor noise from first principles and reduced most vibrational resonance of adjacent walls

Mechanical Team Member

Waterloo Aerial Robotics Group

Sep 2025 – Present

Waterloo, ON

- Designed and fabricated a sub-60 g payload-release mechanism for the Valkyrie competition drone, securing objects of varying size and mass in flight and releasing them at a small target with snag-free, propeller-safe routing
- Designed iterations of empennage structures for Helios, a fixed-wing aircraft evolving toward VTOL capability
- Managed design data in SolidWorks PDM across a multi-hundred-member team, integrating mechanical designs with electrical and autonomy subsystems, to help team win the Pip Rudkin Perseverance Award at AEAC 2026

PROJECTS

Active Suspension Model

Python, ISO8608, Vehicle Dynamics, Vibrations, SolidWorks

- Simulated suspension system of a quarter-car rig by deriving EOM, researching real parameters, implementing skyhook control, generated ISO8608 road profiles, and writing scripts using NumPy, SciPy, and Matplotlib
- Simulated an improvement to ride comfort by 9.6% on ISO8608 class C type roads, and 57.1% during resonance
- Created plots and animations to show the difference from passive to skyhook suspension systems using Matplotlib
- Physically built out full quarter-car rig using SolidWorks and tested dropdown, hold and release, and sinusoidal roads

Double Pendulum Chaos Simulation

Python, SciPy, NumPy, Matplotlib

- Derived the full nonlinear equations of motion from first principles by hand from the Euler-Lagrange equation, obtaining closed-form angular accelerations via Cramer's rule, which allowed physics simulations to run
- Built a numerical simulation using python (NumPy, Matplotlib, and SciPy) demonstrating sensitive dependence and chaotic motion arising from variations in initial conditions as small as $1e-9$ rad
- Computed and visualized a time-to-flip fractal across the initial-condition space using NumPy, SciPy, and Matplotlib

4-DOF Robotic Arm 2.0

SolidWorks, ESP32, C++

- Modeled a 50+ part assembly and validated 8 structural components with FEA against load requirements
- Implemented 4-DOF inverse kinematics, trajectory smoothing, and a web-based control interface using C/C++ running directly on the ESP32 for the built-in WiFi and Bluetooth capabilities
- Wrote Python and MATLAB simulations to verify joint limits and workspace before hardware runs, preventing clashing, tipping, and overheating